

OPERATING SYSTEMS AND SYSTEM PROGRAMMING
SKILL – WEEK 4

Name: S Svara Haasini

Roll No: 2520090170

Task 1: Process Synchronization and Child Process Monitoring Using waitpid()

Question:

Process Synchronization and Child Process Monitoring Using waitpid()

Source Code:

```
fault: haasini@SSH: ~/OSSP_upload/Skill_Week4 /Skill_Week4$ cat > T1.c <<'EOF'
ctrl+alt+1

#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main(void)
{
    pid_t pid;
    int status;

    printf("Parent PID: %d\n", getpid());

    pid = fork();

    if (pid < 0)
    {
        perror("fork");
        return 1;
    }

    if (pid == 0)
    {
        printf("Child PID: %d\n", getpid());
        printf("Child Parent PID: %d\n", getppid());

        printf("Child is executing...\n");
        sleep(2);

        printf("Child completed.\n");
        exit(5);
    }
    else
    {
        printf("Parent created child PID: %d\n", pid);
        printf("Parent waiting for child...\n");

        pid_t result = waitpid(pid, &status, 0);

        if (result == -1)
        EOF return 0;f("Parent continues after child termination.\n")
    }
}
```

Output:

Process Synchronization Output

```
haasini@SSH:~/OSSP_upload/Skill_Week4$ gcc -Wall -Wextra T1.c -o T1
./T1
Parent PID: 620
Parent created child PID: 621
Parent waiting for child...
Child PID: 621
Child Parent PID: 620
Child is executing...
Child completed.
waitpid() returned child PID: 621
Child exited normally.
Child exit status: 5
Parent continues after child termination.
```

Observations:

`fork()` creates the child process and returns the child PID to the parent and 0 to the child. Parent and child have different PIDs (parent: 620, child: 621). The child's PPID (620) corresponds exactly to its parent's PID. The parent calls `waitpid(pid, &status, 0)` with the specific child PID and suspends execution until that child terminates. The child performs its work (sleep and print statements) and terminates with `exit(5)`. `waitpid()` returns the child PID after the child terminates. `WIFEXITED()` confirms the child exited normally (not by signal). `WEXITSTATUS()` retrieves the exit status value 5. The parent resumes and prints the completion message, demonstrating successful parent-child synchronization.

Task 2: PATH Variable and Command Resolution

Question:

To Retrieve PATH Variable, Parse Search Directories, Locate Executables, Verify Permissions, Handle Missing Commands, Test Command Resolution

Source Code:

```

Completion terminated.
haasini@SSH:~/OSSP_upload/Skill_Week4$ cat T2.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

int main(int argc, char *argv[])
{
    if (argc != 2)
    {
        printf("Usage: %s <command>\n", argv[0]);
        return 1;
    }

    char *path = getenv("PATH");
    if (path == NULL)
    {
        printf("PATH variable not found.\n");
        return 1;
    }

    printf("PATH = %s\n\n", path);
    char *path_copy = strdup(path);
    if (path_copy == NULL)
    {
        perror("strdup");
        return 1;
    }

    char *directory = strtok(path_copy, ":");
    char full_path[1024];
    while (directory != NULL)
    {
        snprintf(full_path, sizeof(full_path),
                 "%s/%s", directory, argv[1]);

        if (access(full_path, F_OK) == 0)
        {
            printf("File found: %s\n", full_path);
            if (access(full_path, X_OK) == 0)
            {
                printf("Execute permission: YES\n");
                printf("Command resolved successfully.\n");
            }
            else
            {
                printf("Execute permission: NO\n");
            }

            free(path_copy);
            return 0;
        }

        directory = strtok(NULL, ":");
    }

    printf("Command '%s' not found in PATH.\n", argv[1]);
    free(path_copy);
    return 1;
}

```

Output:

Successful PATH Resolution and Missing Command Handling

```

haasini@SSH:~/OSSP_upload/Skill_Week4$ gcc -Wall -Wextra T2.c -o T2
haasini@SSH:~/OSSP_upload/Skill_Week4$ ./T2 pwd
PATH = /usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/usr/lib/wsl/lib:/mnt/c/Program Files/Common Files/Oracle/Java/javapath:/mnt/c/Program Files (x86)/Common Files/Oracle/Java/javapath:/mnt/c/Program Files (x86)/Common Files/Oracle/Java/javapath:/mnt/c/Windows/system32:/mnt/c/Windows:/mnt/c/Windows/System32/Wbem:/mnt/c/Windows/System32/WindowsPowerShell/v1.0:/mnt/c/Windows/System32/OpenSSH:/mnt/c/Program Files/Git/cmd:/mnt/c/Program Files/no dejs:/mnt/c/ProgramData/chocolatey/bin:/mnt/c/Program Files/MySQL/MySQL Shell 8.0/bin:/mnt/c/Users/haasi/AppData/Local/Microsoft/WindowsApps:/mnt/c/Program Files/Java/jdk-25/bin:/mnt/c/Users/haasi/AppData/Local/Programs/Microsoft VS Code/bin:/mnt/c/Users/haasi/AppData/Local/GitHubDesktop/bin:/mnt/c/Program Files/JetBrains/PyCharm 2026.1/bin:/mnt/c/Users/haasi/.lmstudio/bin:/mnt/c/Users/haasi/AppData/Roaming/npm:/mnt/c/Program Files/JetBrains/PyCharm 2026.1/bin:/mnt/c/Users/haasi/AppData/Local/Python/bin:/mnt/c/Users/haasi/AppData/Local/Kingsoft/WPS Office/12.1.0.27458/clitool:/mnt/c/Users/haasi/AppData/Local/Programs/Antigravity IDE/bin:/snap/bin

File found: /usr/bin/pwd
Execute permission: YES
Command resolved successfully.
haasini@SSH:~/OSSP_upload/Skill_Week4$ ./T2 Raghu
PATH = /usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/usr/lib/wsl/lib:/mnt/c/Program Files/Common Files/Oracle/Java/javapath:/mnt/c/Program Files (x86)/Common Files/Oracle/Java/javapath:/mnt/c/Program Files (x86)/Common Files/Oracle/Java/javapath:/mnt/c/Windows/system32:/mnt/c/Windows:/mnt/c/Windows/System32/Wbem:/mnt/c/Windows/System32/WindowsPowerShell/v1.0:/mnt/c/Windows/System32/OpenSSH:/mnt/c/Program Files/Git/cmd:/mnt/c/Program Files/no dejs:/mnt/c/ProgramData/chocolatey/bin:/mnt/c/Program Files/MySQL/MySQL Shell 8.0/bin:/mnt/c/Users/haasi/AppData/Local/Microsoft/WindowsApps:/mnt/c/Program Files/Java/jdk-25/bin:/mnt/c/Users/haasi/AppData/Local/Programs/Microsoft VS Code/bin:/mnt/c/Users/haasi/AppData/Local/GitHubDesktop/bin:/mnt/c/Program Files/JetBrains/PyCharm 2026.1/bin:/mnt/c/Users/haasi/.lmstudio/bin:/mnt/c/Users/haasi/AppData/Roaming/npm:/mnt/c/Program Files/JetBrains/PyCharm 2026.1/bin:/mnt/c/Users/haasi/AppData/Local/Python/bin:/mnt/c/Users/haasi/AppData/Local/Kingsoft/WPS Office/12.1.0.27458/clitool:/mnt/c/Users/haasi/AppData/Local/Programs/Antigravity IDE/bin:/snap/bin

Command 'Raghu' not found in PATH.

```

Observations:

getenv("PATH") retrieves the PATH environment variable containing all search directories. strdup() creates a modifiable copy because strtok() will tokenize and modify the string in-place. strtok() separates the PATH by colon delimiters into individual directory paths. For each directory, snprintf() constructs the full executable path by concatenating directory + "/" + command name. access(full_path, F_OK) checks whether the candidate file exists in the filesystem. access(full_path, X_OK) verifies execute permission on the file. When searching for "pwd", the program successfully locates /usr/bin/pwd with execute permission and reports "Command resolved successfully." When searching for "Raghu", no matching executable is found in any PATH directory, and the program reports "Command 'Raghu' not found in PATH." free() releases the dynamically allocated PATH copy.

Task 3: Environment Variable Expansion

Question:

To Detect Variable References, Expand Values, Handle Undefined Variables, Update Tokens, Support Nested Variables, Test Expansion Logic

Source Code:

```

haasini@SSH:~/OSSP_upload/Skill_Week4$ cat > T3.c <<'EOF'
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

#define MAX_INPUT 512
#define MAX_OUTPUT 1024

int main(void)
{
    char input[MAX_INPUT];
    char output[MAX_OUTPUT];

    printf("Enter a string: ");

    if (fgets(input, sizeof(input), stdin) == NULL)
        return 1;

    input[strcspn(input, "\n")] = '\0';

    int i = 0;
    int j = 0;

    while (input[i] != '\0' && j < MAX_OUTPUT - 1)
    {
        if (input[i] == '$')
        {
            i++;

            char variable[128];
            int k = 0;

            while ((isalnum((unsigned char)input[i]) ||
                    input[i] == '_' &&
                    k < 127))
            {
                variable[k++] = input[i++];
            }

            EOF return 0;
            expanded : %s\n", output);
            ined variable: %s\n",
haasini@SSH:~/OSSP_upload/Skill_Week4$ gcc -Wall -Wextra T3.c -o T3
haasini@SSH:~/OSSP_upload/Skill_Week4$ ./T3
Enter a string: My home is $HOME

```

Output:

Defined and Undefined Variable Expansion Cases

```

haasini@SSH:~/OSSP_upload/Skill_Week4$ gcc -Wall -Wextra T3.c -o T3
haasini@SSH:~/OSSP_upload/Skill_Week4$ ./T3
Enter a string: My home is $HOME
Original : My home is $HOME
Expanded : My home is /home/haasini
haasini@SSH:~/OSSP_upload/Skill_Week4$ ./T3
Enter a string: hello $abc
Warning: Undefined variable: abc
Original : hello $abc
Expanded : hello

```

Observations:

The program scans the input string for the dollar sign (\$) character. When \$ is detected, subsequent characters are collected and interpreted as an environment variable name using `isalnum()`. `getenv()` retrieves the corresponding environment variable value. When processing "My home is \$HOME", the program successfully expands \$HOME to /home/haasini (the actual home directory on this system). When processing "hello \$abc", `getenv()` returns NULL because abc is not defined in the environment, triggering a "Warning: Undefined variable: abc" message. The undefined variable contributes no text to the expanded output, resulting in "hello". `fgets()` reads user input and `strcspn()` removes the trailing newline. The program demonstrates both successful expansion of defined variables and graceful handling of undefined variables with warnings.

Task 4: Built-in Command Dispatch

Question:

To Identify Built-ins, Create Dispatch Table, Execute In-Process Built-ins, Handle Invalid Commands, Maintain State, Test Dispatch Logic

Source Code:

```
Expanded : netto
haasini@SSH:~/OSSP_upload/Skill_Week4$ cat > T4.c <<'EOF'
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

#define MAX_INPUT 256

void builtin_pwd(void)
{
    char cwd[1024];
    if (getcwd(cwd, sizeof(cwd)) != NULL)
        printf("%s\n", cwd);
    else
        perror("getcwd");
}

void builtin_cd(char *directory)
{
    if (directory == NULL)
        directory = getenv("HOME");
    if (chdir(directory) != 0)
        perror("cd");
}

void builtin_echo(char *text)
{
    if (text != NULL)
        printf("%s\n", text);
    else
        printf("\n");
}

int main(void)
{
    char input[MAX_INPUT];
    while (1)
    {
EOF return 0;rintf("Command not found: %s\n", original);
```

```

haasini@SSH:~/OSSP_upload/Skill_Week4$ cat > T4.c <<'EOF'
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

#define MAX_INPUT 256

void builtin_pwd(void)
{
    char cwd[1024];
    if (getcwd(cwd, sizeof(cwd)) != NULL)
        printf("%s\n", cwd);
    else
        perror("getcwd");
}

void builtin_cd(char *directory)
{
    if (directory == NULL)
        directory = getenv("HOME");
    if (chdir(directory) != 0)
        perror("cd");
}

void builtin_echo(char *text)
{
    if (text != NULL)
        printf("%s\n", text);
    else
        printf("\n");
}

int main(void)
{
    char input[MAX_INPUT];
    while (1)
    {
EOF return 0;rintf("Command not found: %s\n", original);

```

Output:

Built-in Command Execution

```

haasini@SSH:~/OSSP_upload/Skill_Week4$ ./T4
myshell> pwd
/home/haasini/OSSP_upload/Skill_Week4
myshell> echo Hello World
Hello World
myshell> cd /tmp
myshell> pwd
/tmp
myshell> abc
Command not found: abc
myshell> exit
Exiting shell...
haasini@SSH:~/OSSP_upload/Skill_Week4$

```

Observations:

The program implements built-in command dispatch using conditional comparisons (strcmp) to identify built-in command names. pwd executes as an in-process built-in using getcwd() to retrieve and print the current working directory, initially /home/haasini/OSSP_upload/Skill_Week4. echo Hello World prints the supplied text argument directly without spawning a child process. cd /tmp changes the shell process's working directory using chdir(), demonstrating in-process state change. Running pwd after cd /tmp outputs /tmp, proving the directory change persists in the same shell process (not in a child). abc is not a recognized built-in, triggering "Command not found: abc" to demonstrate invalid-command handling. exit terminates the main loop cleanly and exits the

program. `strtok()` separates the input command from its arguments; `strcmp()` compares the command string against known built-in names to dispatch to the appropriate function.